TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TỔ TIÊN CHUNG GẦN NHẤT

Lowest Common Ancestor

Từ cơ bản đến nâng cao

TẬP 3 HƯỚNG DẪN GIẢI

```
up[k][v] = up[k-1][up[k-1][u]]
```

Lê Thành Nam



Tổng hợp và biên soạn

Lê Thành Nam

1 Lý thuyết 2 Bài tập 3 **Hướng dẫn giải**

LỜI NÓI ĐẦU

Cuốn sách này là Tập 3 của bộ tài liệu **Tổ tiên chung gần nhất (LCA): Từ cơ bản đến nâng cao**, gồm hướng dẫn giải cho toàn bộ 36 bài tập của Tập 2. Mỗi lời giải được viết theo cùng một trình tự:

- Tóm tắt đề: Phát biểu lại bài toán bằng ngôn ngữ của cây, gạt bỏ phần "câu chuyện".
- Phân tích: Các nhận xét then chốt, có chứng minh và chạy tay trên ví dụ.
- Từng subtask: Cách làm đủ cho subtask đó, đếm số phép tính để thấy vì sao nó đủ nhanh và vì sao nó không đủ cho subtask sau, kèm chương trình hoàn chỉnh hoặc đoạn code ý chính.
- Bảng điểm: Mỗi cách làm đạt những subtask nào, được bao nhiêu điểm.
- Bẫy thường gặp: Những lỗi dễ mắc trong bài.

Mọi chương trình trong sách đều đã được biên dịch theo chuẩn C++14 và được đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên, gồm cây ngẫu nhiên, đường thẳng, hình sao, hình "con sâu", với số hiệu đỉnh bị xáo trộn. Các chương trình tối ưu đều đã được chạy thử trên dữ liệu lớn nhất để bảo đảm thời gian.

Một lời khuyên: Đừng đọc lời giải như đọc truyện. Hãy đọc phần phân tích tới nhận xét đầu tiên, rồi gấp sách lại và tự làm tiếp. Chỉ mở lại khi đã thật sự bế tắc.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁC ĐOẠN CODE DÙNG CHUNG

Nhiều lời giải dùng lại cùng một vài đoạn code. Để không lặp lại lời giải thích, ta ghi chúng ở đây; mỗi chương trình trong sách vẫn được in **đầy đủ** để em chép và chạy được ngay.

Duyệt cây theo chiều rộng (BFS) để có cha và độ sâu

Code ý chính: BFS từ gốc 1 trên cây cho dạng danh sách cạnh

```
vector<int> bfs;
bfs.push_back(1);
for (int i = 0; i < (int)bfs.size(); i++) {
    int u = bfs[i];
    for (int v : g[u])
        if (v != p[u]) {
            p[v] = u;
            dep[v] = dep[u] + 1;
            bfs.push_back(v);
        }
}
```

Tìm LCA bằng nhảy nhị phân

Code ý chính: up[k][u] là tổ tiên cách u đúng 2^k đời, up[k][u] = 1

```
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int k = 0; k < LOG; k++)
        if (d >> k & 1) u = up[k][u];
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; k--)
        if (up[k][u] != up[k][v]) {
            u = up[k][u];
            v = up[k][v];
        }
    return up[0][u];
}
```

Duyệt theo chiều sâu không đệ quy để có tin, tout

Code ý chính: Mỗi đỉnh có con trỏ it[u] chỉ tới đỉnh kề tiếp theo cần xét

```
tin[1] = ++timer;
st.push_back(1);
while (!st.empty()) {
    int u = st.back();
    if (it[u] < (int)g[u].size()) {
        int v = g[u][it[u]++];
        if (v == p[u]) continue;
        p[v] = u;
        tin[v] = ++timer;
        st.push_back(v);
    } else {
        tout[u] = timer;
        st.pop_back();
    }
}
```

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. NHẬP MÔN TỔ TIÊN CHUNG GẦN NHẤT **6**
Bài 1.1. BACHOHANG: Bậc họ hàng 7
Bài 1.2. CUOCHOP: Cuộc họp dòng họ 12
Bài 1.3. DIEMHEN: Điểm hẹn 16
Bài 2.1. THUMUC: Cây thư mục 23
Bài 2.2. DEMTANG: Đếm theo tầng 26
Bài 2.3. TRENDUONG: Trên đường đi 32
CHƯƠNG 2. BA PHƯƠNG PHÁP TÌM LCA **38**
Bài 3.1. TOTIENK: Tổ tiên thứ k 39
Bài 3.2. TUOI: Người lớn tuổi nhất 42
Bài 3.3. DICHCHUYEN: Cổng dịch chuyển 46
Bài 4.1. BANGTHUA: Bảng thưa 50
Bài 4.2. TONGKC: Tổng khoảng cách 53
Bài 4.3. LCADOAN: LCA của một đoạn 58
Bài 5.1. DEMCAO: Điểm cao nhất 65
Bài 5.2. DUONGDAI: Đường dài 68
Bài 5.3. TOTIENMAU: Tổ tiên cùng màu 73
CHƯƠNG 3. ỨNG DỤNG CỦA LCA **77**
Bài 6.1. KHOANGCACH: Khoảng cách 78
Bài 6.2. GAPNHAU: Gặp nhau 82
Bài 6.3. GIAODUONG: Đoạn đường chung 85
Bài 7.1. CANHMAX: Cạnh yếu nhất, cầu mạnh nhất 89
Bài 7.2. XETAI: Xe tải 92
Bài 7.3. CAYKHUNG2: Cây khung chứa một cạnh 98
Bài 8.1. TONGDUONG: Tổng trên đường đi 104
Bài 8.2. PHUCANH: Đếm lượt qua cầu 107
Bài 8.3. VANCHUYEN: Kế hoạch vận chuyển 110
CHƯƠNG 4. LCA NÂNG CAO **116**
Bài 9.1. NHENHE: Cạnh nhẹ 117
Bài 9.2. DOIDIEM: Đổi điểm 120
Bài 9.3. QTREE: Cầu mạnh nhất 124
Bài 10.1. CHAMOI: Cha mới 129
Bài 10.2. CAYCONGOC: Cây con theo gốc 132
Bài 10.3. GOCDONG: Gốc di động 135
Bài 11.1. STEINER: Mạng kết nối tối thiểu 139
Bài 11.2. GANNHAT: Cặp gần nhất 143
Bài 11.3. CHIACAT: Chia cắt 148
Bài 12.1. XANHAT: Xa nhất 153
Bài 12.2. THEMLA: Thêm lá 156
Bài 12.3. TAPXA: Tập xa nhất 159

Lê Thành Nam

CHƯƠNG 1. NHẬP MÔN TỔ TIÊN CHUNG GẦN NHẤT

Chương này hướng dẫn giải sáu bài tập đi cùng Bài 1 và Bài 2 của Tập 1. Với mỗi bài, ta đi lần lượt qua các subtask: Cách làm nào đủ cho subtask nào, vì sao, và chương trình hoàn chỉnh của cách làm đó. Subtask cuối của các bài 1.1, 1.2, 1.3 cần nhảy nhị phân (Bài 3 của Tập 1); em có thể đọc trước phần lời giải đó như một lời giới thiệu, rồi đọc lại kĩ sau khi học Bài 3.

TRONG CHƯƠNG NÀY

Bài 1.1. BACHOHANG: Bậc họ hàng *(Cơ bản)*
Bài 1.2. CUOCHOP: Cuộc họp dòng họ *(Trung bình)*
Bài 1.3. DIEMHEN: Điểm hẹn *(Nâng cao)*
Bài 2.1. THUMUC: Cây thư mục *(Cơ bản)*
Bài 2.2. DEMTANG: Đếm theo tầng *(Trung bình)*
Bài 2.3. TRENDUONG: Trên đường đi *(Nâng cao)*

Lê Thành Nam

BÀI 1.1 BACHOHANG · Bậc họ hàng Cơ bản

TÓM TẮT ĐỀ

Cho cây gốc 1 dưới dạng mảng cha p[2..n] và q câu hỏi (u, v). Với mỗi câu hỏi, gọi w = LCA(u, v); in a = dep[u] − dep[w] và b = dep[v] − dep[w].

Phân tích

Đây là bài toán LCA "khoác áo" gia phả. Số đời từ u lên w chính là hiệu độ sâu dep[u] − dep[w], vì mỗi đời là một cạnh đi lên. Vậy chỉ cần tính được độ sâu của mọi người và LCA của từng cặp. Ba subtask ứng với ba cách tìm LCA với tốc độ tăng dần.

Một chi tiết cần nhớ: Đề không hứa p[v] < v, nên độ sâu phải tính bằng BFS từ gốc (Tập 1, Bài 1, Bước 5).

Subtask 1 Đánh dấu và xóa toàn bộ (30 điểm)

Với mỗi câu hỏi, đi lên từ u và ghi vào mảng pos[x] = số bước từ u tới x với mọi tổ tiên x của u (các đỉnh khác có pos = −1). Sau đó đi lên từ v, đếm số bước b, cho tới khi gặp đỉnh x có pos[x] ≥ 0. Khi đó x là LCA, a = pos[x]. Cách này không cần mảng độ sâu.

Code ý chính: Subtask 1: mỗi câu hỏi O(n)

```
fill(pos.begin(), pos.end(), -1);
int a = 0;
for (int x = u; x != 0; x = p[x]) pos[x] = a++;
int b = 0, x = v;
while (pos[x] < 0) {
    x = p[x];
    b++;
}
cout << pos[x] << ' ' << b << '\n';
```

ĐẾM SỐ PHÉP TÍNH

Lệnh fill xóa n phần tử, hai vòng đi lên mỗi vòng không quá n bước. Mỗi câu hỏi O(n), tổng $O(n \cdot q) = 10^3 \cdot 10^3 = 10^6$ phép với subtask 1. Với $n = q = 2 \cdot 10^5$, con số này là $4 \cdot 10^{10}$: Quá chậm.

Subtask 2 Cân độ sâu rồi cùng leo (70 điểm)

Tính dep[] bằng BFS. Mỗi câu hỏi: Sao chép u, v sang a, b; kéo đỉnh sâu hơn lên cho bằng tầng; cùng leo cho tới khi gặp nhau. Đỉnh gặp nhau là w. Mỗi câu hỏi tốn O(h), với $h \le 1000$ là chiều cao gia phả.

Lê Thành Nam

BACHOHANG.CPP (C2): Cân độ sâu rồi cùng leo, O(n + q·h)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("BACHOHANG.INP", "r")) {
        freopen("BACHOHANG.INP", "r", stdin);
        freopen("BACHOHANG.OUT", "w", stdout);
    }
    int n, q;
    cin >> n >> q;
    vector<int> p(n + 1, 0), dep(n + 1, 0);
    vector<vector<int>> g(n + 1);
    for (int v = 2; v <= n; v++) {
        cin >> p[v];
        g[p[v]].push_back(v);
    }
    vector<int> bfs;
    bfs.push_back(1);
    for (int i = 0; i < (int)bfs.size(); i++) {
        int u = bfs[i];
        for (int v : g[u]) {
            dep[v] = dep[u] + 1;
            bfs.push_back(v);
        }
    }
    while (q--) {
        int u, v;
        cin >> u >> v;
        int a = u, b = v;
        while (dep[a] > dep[b]) a = p[a];
        while (dep[b] > dep[a]) b = p[b];
        while (a != b) {
            a = p[a];
            b = p[b];
        }
        cout << dep[u] - dep[a] << ' ' << dep[v] - dep[a] << '\n';
    }
    return 0;
}
```

Lê Thành Nam

ĐẾM SỐ PHÉP TÍNH

Với subtask 2: $q \cdot 2h \le 5 \cdot 10^4 \cdot 2 \cdot 10^3 = 10^8$ bước leo, chạy khoảng 0,35 giây. Chương trình cũng qua subtask 1. Ở subtask 3, cây có thể là đường thẳng cao $2 \cdot 10^5$, khi đó $q \cdot h$ lên tới $4 \cdot 10^{10}$: Quá chậm.

Subtask 3 Nhảy nhị phân (100 điểm)

Dựng bảng up[k][u] là tổ tiên cách u đúng 2^k đời (Tập 1, Bài 3). Bảng tính được trong lúc BFS: Khi lấy u ra khỏi hàng đợi, mọi tổ tiên của u đã có hàng trong bảng, nên up[k][u] = up[k − 1][up[k − 1][u]]. Quy ước cha của gốc là chính gốc (up[k][1] = 1) để các bước nhảy "quá tay" dừng ở gốc. Mỗi câu hỏi LCA tốn O(log n).

BACHOHANG.CPP: Nhảy nhị phân, O((n + q) log n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

const int LOG = 18;
int n, q;
vector<int> dep;
vector<vector<int>> up;

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int k = 0; k < LOG; k++)
        if (d >> k & 1) u = up[k][u];
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; k--)
        if (up[k][u] != up[k][v]) {
            u = up[k][u];
            v = up[k][v];
        }
    return up[0][u];
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("BACHOHANG.INP", "r")) {
        freopen("BACHOHANG.INP", "r", stdin);
        freopen("BACHOHANG.OUT", "w", stdout);
    }
    cin >> n >> q;
    dep.assign(n + 1, 0);
    up.assign(LOG, vector<int>(n + 1, 1));
    vector<vector<int>> g(n + 1);
    for (int v = 2; v <= n; v++) {
        cin >> up[0][v];
        g[up[0][v]].push_back(v);
    }
    vector<int> bfs;
    bfs.push_back(1);
    for (int i = 0; i < (int)bfs.size(); i++) {
        int u = bfs[i];
        for (int k = 1; k < LOG; k++) up[k][u] = up[k - 1][up[k - 1][u]];
        for (int v : g[u]) {
            dep[v] = dep[u] + 1;
            bfs.push_back(v);
        }
    }
    while (q--) {
        int u, v;
        cin >> u >> v;
        int l = lca(u, v);
        cout << dep[u] - dep[l] << ' ' << dep[v] - dep[l] << '\n';
    }
    return 0;
}
```

Lê Thành Nam

ĐẾM SỐ PHÉP TÍNH

Bảng up có $18 \cdot (n + 1) \approx 3{,}6 \cdot 10^6$ số nguyên, khoảng 14 MB. Tiền xử lý 18n phép, mỗi câu hỏi không quá $2 \cdot 18$ bước nhảy. Tổng khoảng $18 \cdot 2 \cdot 10^5 + 36 \cdot 2 \cdot 10^5 \approx 10^7$ phép: Chạy dưới 0,15 giây.

Bảng điểm

Cách làm	Độ phức tạp	Subtask đạt	Điểm
Đánh dấu và xóa toàn bộ	$O(n \cdot q)$	1	30
Cân độ sâu rồi cùng leo	$O(n + q \cdot h)$	1, 2	70
Nhảy nhị phân	$O((n + q) \log n)$	1, 2, 3	100

BẪY THƯỜNG GẶP

- Tính dep theo thứ tự v = 2..n với giả thiết p[v] < v: Sai khi cha mang số lớn hơn con.
- In dep[w] − dep[u] (ngược dấu): Kết quả âm.
- Dùng LOG = 17 với $n = 2 \cdot 10^5$: $2^{17} = 131072 < 2 \cdot 10^5$, bước nhảy dài nhất không đủ. Cần $2^{LOG} > n$, tức LOG = 18.

Lê Thành Nam